

THE ZERO PARADOX

ZP-K: Computational Grounding of Self-Reference

Initial: April 2026 | Current: 1.25, October 2026

This document establishes the computational grounding of the Zero Paradox's central self-reference structure. On the framework's reading, $\perp$ in the computational instantiation is not a state of a Turing machine: it is the ground state of a universal Turing machine — the state from which no external executor is required. Kleene's second recursion theorem supplies a fixed point of the self-application operator; that this fixed point is to be read as the computational expression of $\perp = \{\perp\}$ is the framework's commitment, carried by the KleeneStructure class, not a property the theorem establishes.

The central result is a three-way equivalence within this framework, in the precise sense defined by Remark R-K.0: the structural roles of $\perp$ — Quine atom (set-theoretic), bottom element (order-theoretic), and join identity (algebraic) — are PROVED to coincide, while the fourth role, Kleene fixed point (computational), is required as a typeclass field rather than derived. (See Remark R-K.0 for the precise scope: equivalences (1)–(3) are derived via T-EXEC; equivalence (4) is combined by KleeneStructure's typeclass requirement, not derived independently.)

Section I: The Kleene Fixed Point

I. Kleene's Second Recursion Theorem

Kleene's second recursion theorem is the computational fixed-point theorem. For any partially computable transformation f of programs, there exists a program e such that e and $f(e)$ compute the same function. Applied to the identity: there exists a program whose output at any input n equals its output at its own Gödel number plus n — a periodicity fixed point.

In Lean 4, this is formalized in Mathlib's computability library. For any partially computable transformation f of codes, there exists a code c such that $\text{eval } c = f \ c$. The theorem supplies a code without distinguishing one. Section IV tabulates the measured axiom footprints.

Kleene's Second Recursion Theorem (Mathlib)

For any partially computable $f : \text{Code} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, there exists $c : \text{Code}$ such that $\text{eval } c = f \ c$.

Applied to the selfApply transformation, this yields a code c satisfying $\text{eval } c \ n = \text{eval } c \ (\text{encode } c + n)$ for all n — a periodicity condition with period c 's own Gödel number, though not necessarily the least. Multiple such codes exist.

II. The Self-Application Map

The self-application map sends each code c to the partial function that runs c on c 's own Gödel number plus an offset. A fixed point of self-application satisfies a periodicity condition: $\text{eval } c \ n = \text{eval } c \ (\text{encode } c + n)$ for all n , with period equal to c 's own Gödel number, though not necessarily the least. Non-uniqueness is expected: the family of fixed points is infinite, witnessed by the constant codes (`infinite_quine_family`). `Kleene.lean § I` states what that does and does not imply about codes generally.

Definition: selfApply and IsComputationalQuine (Kleene.lean § I)

`selfApply` : `Code` $\rightarrow$ $\mathbb{N} \rightarrow .\mathbb{N} := \text{fun } c \ n \mapsto \text{eval } c \ (\text{encode } c + n)$

A code c is a computational Quine if $\text{eval } c = \text{selfApply } c$, i.e.:

$\forall n, \text{eval } c \ n = \text{eval } c \ (\text{encode } c + n)$

This is a periodicity condition: c 's output at n equals its output at $\text{encode}(c) + n$. The encoding plays the role of the "address" of the program — c 's behavior at n is the same as c 's behavior at its own address plus n .

`selfApply_partrec`: `selfApply` is partially computable.

Proof: via Mathlib computability primitives — `eval` composed with `encode` and `nat_add`. Lean purity: standard foundational axioms only. ✓

Theorem: computational_quine_exists

There exists a computational Quine: $\exists c : \text{Code}, \text{IsComputationalQuine } c$.

Proof: immediate from `kleene_fixed_point_exists` applied to `selfApply`, using `selfApply_partrec`.

Lean purity: standard foundational axioms only. ✓

Note on uniqueness: unlike the AFA Quine atom (unique by the AFA decoration theorem), computational Quines are not unique. Multiple codes can satisfy the fixed-point equation independently. Uniqueness in ZP-K flows from ZP-J T-EXEC (on the set-theoretic side), not from the computational definition.

Section II: KleeneStructure — Bridging AFA and Computation

I. The Bridge

ZP-J's `AFAStructure` typeclass encodes AFA self-containment in type theory: a predicate `selfMem`, AFA uniqueness (`quine_unique`), and the bridge field `bot_self_mem` ($\perp$ is self-containing). T-EXEC follows: the Quine atom equals $\perp$.

ZP-K's `KleeneStructure` extends `AFAStructure` with a computational witness: a code `botCode` satisfying the `selfApply` periodicity condition, whose existence is guaranteed by Kleene's theorem. The typeclass commitment takes AFA self-containment ($\perp = \{\perp\}$) and the Kleene fixed point to be the same structural role stated in two formal languages — this is the motivating claim, not a consequence derived by the theorems.

KleeneStructure Typeclass (Kleene.lean § II)

class KleeneStructure (L : Type*) [ZPSemilattice L] extends AFAStructure L with:

(inherited) selfMem : L → Prop — self-membership predicate

(inherited) quine_unique — AFA uniqueness

(inherited) bot_self_mem — $\perp$ is self-containing

(new) botCode : Code — a code selected from the existence proof; not a distinguished code

(new) botCode_is_quine : IsComputationalQuine botCode — a CLASS FIELD, not a theorem: the periodicity condition $\text{eval } c \ n = \text{eval } c \ (\text{encode } c + n)$, which constant codes also satisfy

(new) bot_self_mem_from_kleene : selfMem $\perp$ — a RESTATEMENT of the inherited AFAStructure field bot_self_mem, adding no content beyond it. Not an implication from the Kleene side.

Any KleeneStructure instance must supply both the AFA witness (bot_self_mem) and the computational witness (botCode with botCode_is_quine). Both are class FIELDS — assumptions discharged at instantiation, not results proved here. They are required together because the framework READS them as one structural fact; that reading is the motivation for the class, and it is not derived within it.

II. Why Not a Bridge Axiom?

The identification between AFA self-containment and Kleene computational fixed points is not asserted as a new axiom. KleeneStructure is a typeclass: any concrete type claiming this identification must discharge it as a proof obligation. The commitment is checked at instantiation, not accepted globally.

The distinction from ZP-J carries over: a freestanding axiom says "trust me." A typeclass field says "prove it for your specific type, or it does not compile." The MachinePhase instance in § V shows how the obligation is discharged concretely.

Section III: T-COMP — The Three-Way Equivalence, and the Bundled Computational Condition

ZP-J T-EXEC established a three-way equivalence: Quine atom (set-theoretic) $\leftrightarrow$ bottom element (order-theoretic) $\leftrightarrow$ join identity (algebraic). T-COMP proves exactly those three equivalent. ZP-K bundles a fourth face — Kleene fixed point (computational) — as a KleeneStructure class field, so all four are present simultaneously in any KleeneStructure lattice, but the fourth is required at instantiation rather than proved equivalent to the others.

Remark R-K.0 — What T-COMP Proves, and What "Four-Way" Meant

The equivalence among (1)–(4) has two distinct sources:

Remark R-K.0 — What T-COMP Proves, and What "Four-Way" Meant

(1)–(3) are equivalent by T-EXEC (ZP-J): any element that is a Quine atom is also $\perp$ and a join identity, and vice versa. This is a genuine logical derivation — the three properties are proved to coincide from the AFAStructure axioms.

(4) is present in any KleeneStructure instance because KleeneStructure requires it as a typeclass field: `botCode_is_quine` must be supplied at instantiation. There is no independent proof that satisfying condition (1) (being a Quine atom in the AFA sense) entails satisfying condition (4) (being a Kleene fixed point), or vice versa. The two are combined by the typeclass definition — they are required together because we take them to be the same structural fact, not because one is derived from the other.

Stated at the level of types, which is where the gap is visible: conditions (1)–(3) are properties of one element q of the lattice L , and T-COMP relates them to each other. Condition (4) is a property of a `Code`. There is no function, and no equivalence, between `Code` and L anywhere in the development — so the fourth condition is not connected to the other three by a mapping; it is required alongside them by the class. `da1_paths_unified` records exactly this and no more: it is a conjunction of a fact about `bot` in L and a fact about `botCode` in `Code`.

In short: "four-way equivalence" means "all four hold in any KleeneStructure instance." (1)–(3) are independently proved equivalent. (4) is bundled in by the typeclass requirement. The philosophical claim — that Kleene computational self-reference and AFA set-theoretic self-reference are the same thing — is the motivation for the typeclass design, not a consequence derived within it.

Theorem T-COMP — Computational Grounding (Kleene.lean § III)

In any KleeneStructure lattice L , for any $q : L$, the following are equivalent:

(1) `IsQuineAtom q` — set-theoretic self-reference (AFA)

(2) $q = \perp$ — order-theoretic minimum (ZP-A)

(3) $\forall x : L, \text{join } q \ x = x$ — algebraic generator (ZP-A A4)

Separately, the KleeneStructure requirement: $\exists \text{ botCode} : \text{Code}, \text{IsComputationalQuine botCode}$ (computational self-reference). It is not a clause of `t_comp`; it is present in any KleeneStructure instance because `botCode_is_quine` is a required field. The equivalence of (1)–(3) is derived by T-EXEC.

Lean: `ZeroParadox.t_comp`. Purity: standard foundational axioms — from Mathlib computability. ✓

I. Why Four Languages?

DA-1's three informal paths (Path 1: AFA structural, Path 2: informational, Path 3: Kolmogorov/computational) were previously understood as three separate corroborations converging on the same conclusion. In ZP-E each argues for DA-1's precondition, which is what the occurrence commitment asserts, and none derives it. Path 1's Lean counterpart is `da1_closed_concrete`, which proves `IsQuineAtom` ($\perp : \text{MachinePhase}$) and nothing computational; Path 3's Lean counterpart is the `machinePhaseKleene` instance's `botCode_is_quine` field, a KleeneStructure requirement that constant codes also meet, not a witness of execution and not a second independent proof.

Path 1 says: nothing external to $\perp$ can execute $\perp$, so if $\perp$ executes at all, the executor is $\perp$ itself, read as $\perp = \{\perp\}$; that rules out an external executor, not an inert $\perp$. Path 3 says: no shorter external program generates $\perp$; that rules out a shorter external generator, not an inert string, so executing is not derivable from incompressibility. The framework READS these as one claim in two vocabularies. What is proved is weaker and is a conjunction, not an identity: both hold together in any KleeneStructure. They cannot be equated formally — one is a statement about an element of the lattice, the other about a Code, and an equation across those types is not a well-formed proposition.

Theorem: da1_paths_unified (Kleene.lean § IV)

In any KleeneStructure lattice:

$\text{IsQuineAtom } \perp \wedge \text{IsComputationalQuine botCode}$

A CONJUNCTION: the AFA self-containment argument (Path 1) and the Kleene computational fixed-point argument (Path 3) both hold, simultaneously witnessed by the KleeneStructure instance. That they are the SAME fact is a framework reading, not a clause of this theorem — the second conjunct is the class field `botCode_is_quine`.

Lean: `ZeroParadox.da1_paths_unified`. Purity: standard foundational axioms only. ✓

II. The Description-Instantiation Gap

The "description-instantiation gap" asks whether mathematical self-reference implies computational execution: whether a self-referential $\perp$ is running (Sense B, ZP-E DA-1 insert § II) rather than an inert description (Sense A).

On the framework's reading, $\perp$ in the computational instantiation is read as the universal Turing machine in its ground state, which is not a description awaiting an external executor: if it executes at all, it executes itself (Path 1, the framework's requirement, not a theorem). That rules out an external executor, not an inert $\perp$: on `MachinePhase`, $\perp$ is the Quine atom (`da1_closed_concrete`), and T-SNAP holds in a dynamics in which nothing leaves c_0 (`tsnap_holds_but_nothing_moves`). That it does execute, so that the configuration reaching P_0 is running, is the occurrence commitment, which DA-1 consumes and does not supply (ZP-E).

Theorem: description_instantiation_gap_closed (Kleene.lean § IV)

In any KleeneStructure lattice:

$\text{IsQuineAtom } \perp \wedge \forall q : L, \text{IsQuineAtom } q \rightarrow q = \perp$

On the framework's reading, $\perp$ is not a description awaiting an external interpreter: if it executes, it is its own executor, the universal machine in its ground state. Lean proves only the Quine-atom statement above; that $\perp$ executes is the occurrence commitment, not this theorem. Lean witnesses $\perp$ as the AFA Quine atom of `MachinePhase` (`da1_closed_concrete`) and carries the Kleene quine as a `KleeneStructure` requirement (`botCode_is_quine`); that these are one structural fact is the framework's reading.

Lean: `ZeroParadox.description_instantiation_gap_closed`. Purity: standard foundational axioms only. ✓

Section IV: Axiom Footprint

This section records measurements, not a rule. ``#print axioms`` traverses a declaration’s statement as well as its proof term, and reports what a proof used — never what a theorem needs. The following were measured on 2026-09-19 against the pinned Mathlib; re-run them rather than citing this table if the answer matters.

Declaration	Axiom footprint
Nat.Partrec.Code	none
Nat.Partrec.Code.eval	none
IsKleeneFixedPoint	none
encodeCode_self(encodeCode c = encodeCode c, by rfl)	none
encode_self(Encodable.encode c = Encodable.encode c, by rfl)	propext, Classical.choice, Quot.sound
kleene_fixed_point_exists	propext, Classical.choice, Quot.sound
roger_fixed_point_exists	propext, Classical.choice, Quot.sound
machinePhaseKleene	propext, Classical.choice, Quot.sound
da1_closed_concrete	propext, Classical.choice, Quot.sound
machinePhaseAFA	none
bot_is_quine_atom	none
t_exec	none

Two readings of that table are worth naming, and neither generalises beyond the rows above. The two spellings of a code’s Gödel number differ: the raw `encodeCode` is axiom-free, while `Encodable.encode` reaches through Mathlib’s `Denumerable Code` instance, whose own term spends `Classical.choice`. And one statement is proved both ways in `Kleene.lean §V` — `IsQuineAtom (bot : MachinePhase)` via `machinePhaseKleene` carries the triple, and the same statement via `machinePhaseAFA` carries nothing. The axioms do not enter through `ZPSemilattice` or `AFAStructure`.

ZP-J T-EXEC and all its corollaries remain axiom-free. The classical axioms are entirely localised to the computational layer.

Remark: Classical Choice in Computability

Where the measurements above show `Classical.choice`, it is reached through Mathlib’s numbering of program codes (`Denumerable Code`): it is present even in a computable instance with a constant code. ZP-K’s `machinePhaseKleene` also picks `botCode` with `Classical.choose`, which is what makes that instance noncomputable. Whether the numbering’s footprint is essential is not measured. Kleene’s theorem is an existence result, and the code it supplies is selected non-constructively. This is standard in computability theory — the theorem guarantees existence without giving a canonical construction.

The `MachinePhase` instance (§ V) uses `Classical.choose` to pick `botCode` from the existence proof. This makes `machinePhaseKleene` noncomputable, which is correct and expected.

Section V: MachinePhase Instance — DA-1: what Lean witnesses

I. The Concrete Instantiation

ZP-E's MachinePhase type is the two-element type {initial, running} carrying the ZPSemilattice instance (bot = initial = c_0 , join = binary maximum). ZP-J gave it AFAStructure via the selfMem predicate. ZP-K gives it KleeneStructure by adding the computational witness botCode.

The selfMem definition for MachinePhase is: selfMem $x := x = \perp$. This is the CIC-compatible encoding of AFA self-containment: "self-containing" means "equals the bottom element." Anti-foundation is not required at the typeclass level — the relevant structural fact ($\perp$ is the unique self-containing element) is captured by the definition and proved by rfl.

AFAStructure MachinePhase Instance (Kleene.lean § V)

instance machinePhaseAFA : AFAStructure MachinePhase where

selfMem $x := x = \perp$ (self-containing = equals initial state)

quine_unique _ _ $hx hy := hx.trans hy.symm$ (if $x = \perp$ and $y = \perp$ then $x = y$)

bot_self_mem := rfl ($\perp = \perp$, proved by reflexivity)

This is the CIC encoding of $\perp = \{\perp\}$: the initial machine state is self-containing and is the unique element with this property. No ZF+AFA axiom is added — the structural fact is encoded as a definition that Lean verifies.

KleeneStructure MachinePhase Instance (Kleene.lean § V)

noncomputable instance machinePhaseKleene : KleeneStructure MachinePhase where

botCode := Classical.choose computational_quine_exists

botCode_is_quine := Classical.choose_spec computational_quine_exists

bot_self_mem_from_kleene := rfl

botCode is selected non-constructively from the existence proof provided by Kleene's theorem — an arbitrary witness, not a distinguished code. Reading it as "the program that IS its own program" is the framework's commitment carried by the class, not a property this instance establishes: IsComputationalQuine is a periodicity condition, strictly weaker than self-reference.

II. DA-1: what Lean witnesses

With the MachinePhase KleeneStructure instance in place, the abstract theorem da1_computational (which holds for any KleeneStructure lattice) applies directly to ZP-E's machine. The result is concrete.

Theorem `da1_closed_concrete` — Path 1's Lean counterpart (Kleene.lean § V)

`da1_closed_concrete` : `IsQuineAtom` ($\perp$: `MachinePhase`)

The initial machine state c_0 is a Quine atom: it is self-containing and is the unique self-containing element of the `MachinePhase` lattice.

Interpretation (the framework's reading, not a Lean theorem): c_0 is read as the universal Turing machine in its ground state, needing no external executor: if it executes at all, it executes itself (Path 1). That it executes is the occurrence commitment, which DA-1 consumes, not a consequence of this theorem.

Lean: `ZeroParadox.da1_closed_concrete`. Purity: standard foundational axioms only (the same statement has an axiom-free proof, `bot_is_quine_atom`). ✓

III. What Changed for DA-1

ZP-E's DA-1 section previously carried the designation "Outside Lean Scope" with three justifications: Path 1 requires ZF+AFA (incompatible with Lean's CIC/MLTT); Path 3 requires Kolmogorov complexity (uncomputable, absent from Mathlib); Path 2 requires a step from unbounded surprisal to executing that no proof in type theory supplies.

For Path 1 (AFA structural), the Lean counterpart is `da1_closed_concrete`, which proves `IsQuineAtom` ($\perp$: `MachinePhase`) and nothing computational: in the `AFAStructure` typeclass, `selfMem` encodes $\perp = \{\perp\}$ in CIC-compatible form, and the proof obligation is discharged by the `MachinePhase` instance. Path 3 (computational) is NOT resolved here. Its Lean counterpart, `botCode_is_quine`, is a `KleeneStructure` class field — an assumption supplied at instantiation, not a second independent proof — and the condition it requires, `IsComputationalQuine`, is a periodicity condition that a constant code satisfies trivially. The Kolmogorov reading ("no shorter program is prior to $\perp$ ") has no formal content in ZP-K: Kolmogorov complexity is uncomputable and absent from the development. Path 3 therefore derives nothing here: in ZP-E it bears on DA-1's precondition and shows that executing is not derivable from incompressibility, and that precondition is what the occurrence commitment asserts.

Path 2 (informational: unbounded surprisal $\rightarrow$ executing) is a bridge principle of its own — a missing principle, not a missing proof — which the framework does not adopt, which is not the occurrence commitment, and which is not a premise of the Snap. The mathematics of L-INF (l_{inf}) is proved; but the step from "exceeds every finite informational bound" to "therefore executing" asks what it means for a mathematical structure to instantiate rather than merely satisfy conditions. No computability library answers this question. Importantly, DA-1 does not depend on Path 2: the formal spine (`DP-2` + `da1_minimal_path`) is proved axiom-free. Path 2 is motivational context.

DA-1 Lean scope status after ZP-K: Path 1 (structural, AFA): Lean counterpart `da1_closed_concrete : IsQuineAtom ( $\perp$  : MachinePhase)`, nothing computational; it rules out an external executor, not an inert $\perp$. Path 3 (computational, Kleene): `botCode_is_quine` is a `KleeneStructure` class field, assumed at instantiation; Path 3 shows executing is not derivable from incompressibility. Path 2 (informational, L-INF): UNADOPTED BRIDGE PRINCIPLE — a missing principle, not a missing proof, and not the occurrence commitment. None derives DA-1's precondition, which is what the occurrence commitment asserts.

Traceability Register — ZP-K

Claim	Grounded In	Axioms	Status
<code>selfApply_partrec</code>	<code>eval_part</code> (Mathlib) + <code>Primrec.encode</code> + <code>Primrec.nat_add</code>	Standard Mathlib foundational axioms	Lean: <code>selfApply_partrec</code> ✓
<code>computational_quine_exists</code>	<code>kleene_fixed_point_exists</code> + <code>selfApply_partrec</code>	Standard Mathlib foundational axioms	Lean: <code>computational_quine_exists</code> ✓
T-COMP: three-way equivalence	ZP-J T-EXEC (<code>t_exec_triple_iff</code>)	Standard Mathlib foundational axioms	Lean: <code>t_comp</code> ✓
<code>da1_paths_unified</code>	<code>bot_is_quine_atom</code> + <code>botCode_is_quine</code>	Standard Mathlib foundational axioms	Lean: <code>da1_paths_unified</code> ✓
<code>description_instantiation_gap_closed</code>	<code>bot_is_quine_atom</code> + ZP-J <code>t_exec</code>	Standard Mathlib foundational axioms	Lean: <code>description_instantiation_gap_closed</code> ✓
<code>machinePhaseAFA</code> (AFAStructure)	<code>selfMem := x = $\perp$</code> ; <code>quine_unique</code> ; <code>bot_self_mem := rfl</code>	No axioms	CIC encoding of $\perp = \{\perp\}$ for <code>MachinePhase</code> ✓
<code>machinePhaseKleene</code> (KleeneStructure)	<code>machinePhaseAFA</code> + <code>Classical.choose</code> <code>computational_quine_exists</code>	Standard Mathlib foundational axioms	noncomputable — classical choice for <code>botCode</code> ✓
<code>da1_closed_concrete</code>	<code>da1_computational</code> + <code>machinePhaseKleene</code>	Standard Mathlib foundational axioms	Lean: <code>da1_closed_concrete</code> ✓ DA-1 structural half — no Code, no execution (definitionally: under <code>selfMem x := x = $\perp$</code> , reduces to $(\perp = \perp) \wedge (\forall x, x = \perp \Rightarrow x = \perp)$; structural closure by typeclass design — see R-K.0)

Open Items Register — ZP-K

Item	Status	Description
DA-1 Path 1 (AFA structural)	Lean counterpart proved; derives no execution	IsQuineAtom ($\perp$: MachinePhase) — the structural half, Path 1's Lean counterpart. The theorem mentions no Code and no execution. Path 1 says that if $\perp$ executes, it executes itself (the framework's requirement, not a theorem): it rules out an external executor, not an inert $\perp$. That $\perp$ executes is the occurrence commitment, which DA-1 consumes; Path 1 does not close it.
DA-1 Path 3 (computational)	ARGUES FOR THE PRECONDITION — derives nothing	botCode_is_quine is a KleeneStructure class field — assumed at instantiation, not proved. IsComputationalQuine is a periodicity condition satisfied by constant codes, and no Kolmogorov content exists in ZP-K. In ZP-E, Path 3 shows that executing is not derivable from incompressibility; DA-1's precondition is what the occurrence commitment asserts.
DA-1 Path 2 (informational)	UNADOPTED BRIDGE PRINCIPLE	L-INF (l_{inf}) is proved. The bridge "unbounded surprisal $\rightarrow$ executing" is a bridge principle of its own, a missing principle, not a missing proof; the framework does not adopt it, and it is not the occurrence commitment or a premise of the Snap. The gap between 'system at P_0 ' and 'system is running' cannot be closed by any computability library. DA-1 does not depend on Path 2.
selfApply uniqueness	CLOSED — not attempted (correct)	Computational quines are not unique in general. Uniqueness flows from ZP-J T-EXEC (set-theoretic side). No uniqueness theorem for computational quines is needed or appropriate.
Rogers' fixed-point theorem	CLOSED — roger_fixed_point_exists	For any computable $f : \text{Code} \rightarrow \text{Code}$, $\exists c, \text{eval}(f\ c) = \text{eval}\ c$. Lean: roger_fixed_point_exists — standard foundational axioms only. ✓
ZP-B MachinePhase instance	OPEN — future work	The 2-adic model from ZP-B (Q_2 structure) has not been given a KleeneStructure instance. This is a natural extension but not required for T-SNAP or DA-1.

End of ZP-K | Computational Grounding of Self-Reference | DA-1 structural half: da1_closed_concrete : IsQuineAtom ($\perp$: MachinePhase) - no code, no execution | Four-way equivalence (R-K.0): Quine atom, $\perp$ and join identity proved to coincide; the Kleene fixed point is required by the typeclass, not derived | Path 2: an unadopted bridge principle, not a missing proof | All Kleene.lean theorems verified. Axioms: standard Mathlib foundational axioms.